

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 09 · LECTURE NOTES

Passwords and Key Derivation

Where slow is the security property.

MODULE OUTCOMES

- Explain why password hashing differs from all other hashing.
- Explain salts, peppers, and what each prevents.
- Choose and parameterise a password hash.
- Distinguish password hashing from key derivation.
- Design a credential store and its migration.

1 Why slow

PASSWORDS HAVE FAR TOO LITTLE ENTROPY TO RESIST A FAST HASH

A human-chosen password carries perhaps 20 to 40 bits of real entropy — not the 128 bits a generated key would have — because people choose from a small, heavily skewed distribution. **So the search space is small enough to enumerate exhaustively.**

And a fast hash is precisely what lets the adversary enumerate it. Commodity GPUs compute billions of SHA-256 hashes per second, and purpose-built hardware does considerably better — which exhausts a 40-bit space in minutes and a 30-bit space instantly.

So the defence is to make each individual guess expensive. A hash that takes 100 milliseconds reduces billions per second to roughly ten per second per core — a factor of about a hundred million, which converts minutes into centuries for the same password.

Which makes this the only situation in all of cryptography where slowness is the security property — and **why every hash in Module 05 §2's first group is exactly the wrong choice**, having been designed and optimised for the opposite goal.

- **A salt is a unique random value per password, stored alongside the hash, and it need not be secret or even unpredictable** — only unique (Module 02 §4).
- **It prevents precomputation and cross-user comparison: no precomputed table can cover all salts, and two users who chose the same password now have different stored hashes**, so a breach does not reveal which accounts share a password — which is itself valuable information to an attacker.
- **A pepper is a single secret value added to every password, stored separately from the database** — in a key management service or an environment secret (CSCE 701 Module 07) — **so that a database-only breach yields hashes that cannot be attacked at all. It is cheap and underused.**

- **The work factor is the tunable cost parameter**, and **it must be raised over time** as hardware improves — which means it is an operational commitment rather than a one-time decision.
- **And a memory cost is what actually resists GPUs**, which is the specific advance that Argon2 and scrypt make over bcrypt. **GPUs have enormous parallel compute and quite limited memory per execution unit**, so a function requiring substantial memory per evaluation narrows their advantage sharply — and that is the current state of the art.

2 Choosing, and parameterising

USE, IN ORDER OF PREFERENCE

Argon2id	memory-hard, the current recommendation
scrypt	memory-hard, older, well understood
bcrypt	fine, widely available, no memory hardness, and silently TRUNCATES its input at 72 bytes
PBKDF2	acceptable only where a standard requires it; the weakest of these against GPUs

NEVER

SHA-256, SHA-3, BLAKE, MD5, or any fast hash
 a fast hash iterated by hand -- you will get the construction wrong, and it remains GPU-friendly even when you do not

SETTING PARAMETERS

target 100-500 ms on YOUR production hardware, then raise it annually. Measure it; do not copy a number from a blog post written for different hardware.

bcrypt's 72-byte truncation is a real and surprising trap. A long passphrase is silently cut at 72 bytes, so every character beyond that contributes exactly nothing — and no error is raised, which means a user who carefully chose a 100-character passphrase has a 72-byte one. **And the measure-don't-copy instruction is the one most often ignored:** a parameter set appropriate for 2015 server hardware is substantially too weak now, and parameters copied from documentation are frequently the library's conservative minimum rather than a recommendation.

THE PARAMETER IS A BUDGET DECISION, SO STATE IT THAT WAY

A higher work factor costs you CPU time on every single login and costs the adversary proportionally more per guess — so the choice is a direct, quantifiable trade between your login throughput and the attacker's guessing rate.

Which makes your side measurable: at 100 ms per hash on four dedicated cores you support roughly forty logins per second, and you can state precisely what doubling the work factor costs in additional capacity at your peak login rate.

And the adversary's side is measurable too: state the guesses per second a GPU achieves against your actual parameters, and therefore how long a 30-bit password survives an offline attack on a stolen database. Both numbers are obtainable in an afternoon.

Which turns 'we should use stronger password hashing' from an opinion into a defensible number with a stated cost — and is CSCE 701 Module 12 §2's capability-metric argument applied to a single parameter, which is the form in which it is easiest to win.

3 Key derivation, which is a different problem

	Password hashing	Key derivation (HKDF)
Input	Low entropy, human-chosen — 20 to 40 bits.	High entropy already — a shared secret from a key exchange, or a random master key.
Goal	Make each guess expensive.	Spread existing entropy into several independent keys.
Speed	Deliberately slow.	Fast is correct — there is nothing to brute-force.
Output	Stored, for later comparison against a login attempt.	Used immediately as keys, and never stored.
Use	Argon2id.	HKDF.

The error is asymmetric, which is what makes the distinction worth remembering. Using HKDF on a password is a vulnerability — fast derivation over a low-entropy input means the password is brute-forceable — whereas using Argon2 to split a high-entropy session secret is merely slow and wasteful. So when in doubt, the slow choice is the safe mistake, which is a useful asymmetry to carry.

- **Deriving several independent keys from a single shared secret** — which is exactly what TLS does after the handshake completes (Module 08 §1), producing separate keys per direction and per purpose.
- **With a distinct context label for each key**, so that the encryption key and the authentication key are independent and cannot be confused or interchanged.
- **Which removes a genuine hazard: using a single key for two different purposes has produced real breaks**, because the security proofs of two constructions do not generally compose when they share a key — and HKDF makes the separation nearly free, so there is no reason not to.
- **And it needs no slowness at all**, because the input already carries full entropy and there is consequently nothing to brute-force — which is the whole content of the distinction above.
- **So: Argon2id for anything derived from a password, and HKDF for everything derived from a real secret. The per-purpose context label is the habit worth forming** — it costs one string literal and prevents an entire class of key-reuse error.

4 The credential store

MIGRATING A CREDENTIAL STORE WITHOUT KNOWING THE PASSWORDS

You cannot simply rehash the existing passwords, because you do not have them — which is, of course, the entire point of having hashed them. So a migration cannot be a batch job.

So rehash on next successful login: verify the submitted password against the old scheme, and if it verifies, immediately compute and store the new hash while you briefly hold the plaintext in memory. **This is the standard technique and it works well.**

And record the scheme per record, so that both verification paths coexist during the transition — **which means your stored format needs a scheme identifier from the very beginning**, and adding one retroactively is the painful version of this migration.

For accounts that never log in again, eventually force a reset — and **the migration is complete only when the old scheme is removed from the code**, which is the step usually left undone, leaving a weak verification path available indefinitely. **An unfinished migration is a retained vulnerability**, and it belongs on the project plan with a date.

- **A scheme identifier, the parameters used, the salt, and the hash** — all in a single field, which is exactly what the standard encoded formats (the modular crypt format, or PHC strings) give you for free. **Use the library's encoded output rather than splitting it into columns.**
- **Constant-time comparison of the result** (Module 04 §1), which the library's own verify function already handles correctly — so use it rather than comparing hashes yourself.
- **Rate limiting and lockout on the login path**, because **a slow hash protects a stolen database and does essentially nothing against online guessing** — the online adversary is limited by your server, not by the hash cost (CSCE 701 Module 02 §4).
- **No password length cap below something generous, and no composition rules** — which is the evidence-based guidance that NIST adopted after the composition rules were shown to reduce entropy by making choices predictable (CSCE 701 Module 11 §1).
- **And a check against known-breached passwords**, which blocks the credentials actually used in credential stuffing — far more effective per unit of user friction than any composition rule. **The online-versus-offline distinction is essential throughout: hashing cost addresses an offline adversary holding your database, and rate limiting addresses an online one**, and you need both because they are different adversaries.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
RFC 9106 — Argon2 (free)	§2's recommendation in the original , including the parameter selection guidance and the variant rationale.

Resource	What it covers for this module
OWASP — Password Storage Cheat Sheet (free)	§2 and §4 as current practice , maintained and updated as hardware moves — which matters for the parameters.
RFC 5869 — HKDF (free)	§3's construction , short, with the extract-then-expand rationale clearly stated.
NIST SP 800-63B, the memorized secret section (free)	§4's evidence-based guidance — and notable as a standard that reversed long-standing advice on the basis of measurement.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Estimate the entropy** of five passwords you have seen chosen in practice.
- 2 **Measure your hardware's SHA-256 rate** and compute how long a 35-bit space takes.
- 3 **Measure Argon2id at your chosen parameters** and recompute.
- 4 **Tune your parameters to 250 ms** on your actual production hardware.
- 5 **Compute your login throughput** at those parameters, and at double.
- 6 **Demonstrate bcrypt's 72-byte truncation** with two passwords differing only after byte 72.
- 7 **Add a pepper** to a test credential store and verify a database-only copy is unusable.
- 8 **Use HKDF to derive two labelled keys** from one secret, and confirm they differ.
- 9 **Implement rehash-on-login** for a scheme migration, with a scheme identifier.
- 10 **Write the plan for removing the old path**, with a date.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can a fast hash not protect passwords? Give the arithmetic.
- 2 Why is slowness the security property only here?
- 3 What does a salt prevent, and does it need to be secret?
- 4 What does a pepper add, and where is it stored?
- 5 Why does memory hardness resist GPUs?
- 6 Name four password hashes in order, and bcrypt's trap.
- 7 How do you set the work factor, and how do you argue for it?
- 8 Give five differences between password hashing and key derivation.

- 9 Why is the error asymmetric?
- 10 How do you migrate a credential store, and when is it done?